Project Report: Task Management Application (Todo App)

Table of Contents

1. Introduction and Goals
2. Description of Features
3. ER Diagram of the Database
4. Explanation of Code Structure (OOP and Diagrams)
5. Screenshots of Key Pages
6. Challenges and Solutions
7. Conclusion
8. References

1.Introduction and Goals

Introduction

The "Task Manager" project is a web application designed to help users manage their tasks efficiently. It allows users to create, edit, delete tasks, attach files to them, and automatically categorize tasks. The application is built using the Flask framework with a SQLite database, and the interface leverages Bootstrap for a user-friendly experience.

Goals

- Develop a functional task management application.
- Implement user registration, login, and task management with category support.
- Enable file uploads for tasks with size and type restrictions.
- Utilize automatic task categorization using a machine learning model.
- Ensure security (CSRF protection, session management) and a user-friendly interface.

2.Description of Features

The " Todo App " application provides the following features:

Authentication

- **Registration**: Users can create a new account by providing a username and password (page: register.html).
- **Login**: Users can log into the system by entering their credentials (page: login.html). Passwords are verified using hashes.
- **Logout**: Users can end their session, redirecting them to the login page.

Task Management

- **View Task List**: The main page (tasks.html) displays a table of the user's tasks with options to filter by category and search by title.
- **Create Task**: Users can create a new task using a form (task_form.html), specifying a title, description, category, and optional file attachment.

- **Edit Task**: Users can update an existing task using the same form.
- **Delete Task**: Users can delete a task with a confirmation prompt (via a button in the task table).

File Uploads

- Users can attach files to tasks.
- Restrictions: Maximum file size is 5 MB; allowed file types are png, jpg, jpeg, gif, and pdf.
- Files are saved in the app/uploads directory, and their paths are stored in the database.

Task Categorization

- If a user does not manually select a category, the task is categorized automatically.
- A machine learning model (facebook/bart-large-mnli) is used to classify tasks into categories: Study, Work, Personal Goals, Health, Household.
- If categorization fails, the task is assigned the category "Not Defined."

3.ER Diagram of the Database

The application's database consists of two tables: User and Task. Below is their description:

Tables		
task		
id	INTEGER	"id" INTEGER NOT NULL
title	VARCHAR	"title" VARCHAR NOT NULL
description	TEXT	"description" TEXT
status	BOOLEAN	"status" BOOLEAN
file_path	VARCHAR	"file_path" VARCHAR
user_id	INTEGER	"user_id" INTEGER NOT NULL
category	VARCHAR	"category" VARCHAR
user		
id	INTEGER	"id" INTEGER NOT NULL
username	VARCHAR	"username" VARCHAR NOT NULL
password	VARCHAR	"password" VARCHAR NOT NULL

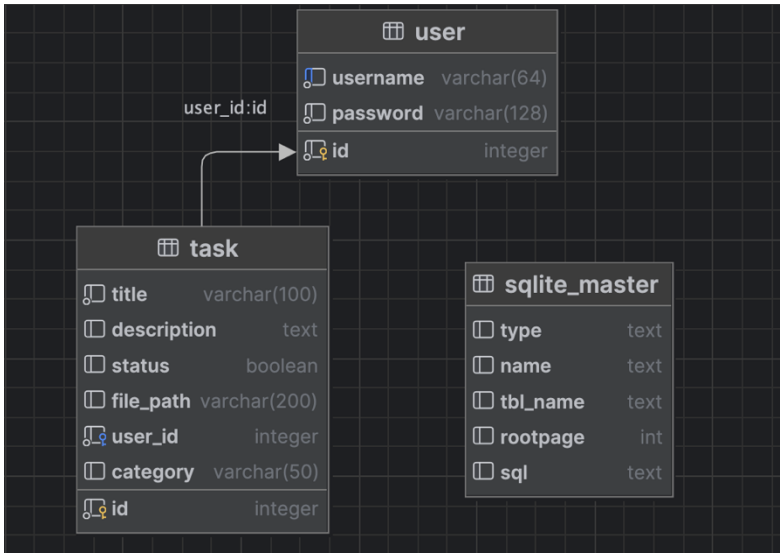
- **Table User**: Stores user data.
  - id (integer, primary key)
  - username (string, unique username)
  - password (string, hashed password)
- **Table Task**: Stores task data.

- id (integer, primary key)
- title (string, task title)
- description (text, task description)
- status (boolean, task completion status)
- file_path (string, path to attached file)
- user_id (integer, foreign key referencing User)
- category (string, task category)

Relationship: A single user (User) can have multiple tasks (Task), establishing a 1-to-many relationship. This is implemented via the user_id field in the Task table, which references the id field in the User table.

Textual Description of the ER Diagram:

- Entity User with attributes: id, username, password.
- Entity Task with attributes: id, title, description, status, file_path, user_id, category.
- A 1:N (one-to-many) relationship exists between User and Task.



4. Explanation of Code Structure (OOP and Diagrams)

Object-Oriented Programming (OOP)

The project employs OOP through models and forms defined in models.py:

- **Class User** (in models.py):

  Fields: id, username, password.

  - set_password: Generates a password hash and stores it.
  - check_password: Verifies if the entered password matches the hash.

- **Class Task** (in models.py):

  - Fields: id, title, description, status, file_path, user_id, category.
  - Relationship: Linked to User via user_id (1-to-many).

- **Class TaskForm** (in models.py):

  - A form for creating and editing tasks.
  - Fields: title (title), description (description), category (category selection), file (file upload), submit (submit button).

The application logic is modularized using Flask Blueprints in routes.py:

- auth_bp: Handles authentication routes (registration, login, logout).
- tasks_bp: Manages task-related routes (list, create, edit, delete)

Textual Description:

- Class User:
  - Attributes: id (Integer), username (String), password (String).
  - Methods: set_password(), check_password().
- Class Task:
  - Attributes: id (Integer), title (String), description (Text), status (Boolean), file_path (String), user_id (Integer), category (String).
- Class TaskForm:
  - Attributes: title (StringField), description (TextAreaField), category (SelectField), file (FileField), submit (SubmitField).
- Relationship: User has a 1:N relationship with Task (via user_id).

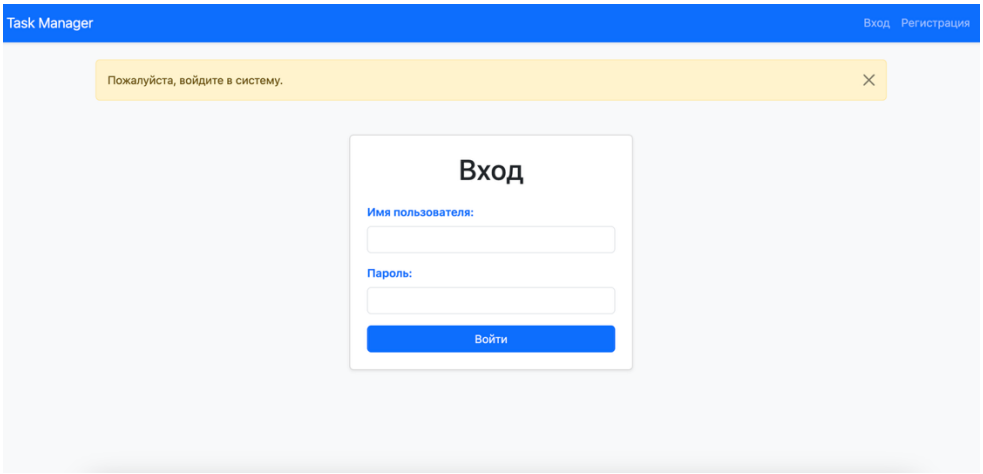
Project Structure:

- app.py: Initializes the Flask application, sets up the SQLite database using SQLAlchemy and Flask-Migrate.
- models.py: Defines the User, Task, and TaskForm classes.
- routes.py: Contains routes and logic, split into two Blueprints: auth_bp and tasks_bp.
- templates/: HTML templates:
  - base.html: Base template with navigation and Bootstrap.
  - login.html: Login page.
  - register.html: Registration page.
  - task_form.html: Form for creating/editing tasks.
  - tasks.html: Task list with filters.
    - Entry point to run the application (app.run())

5.Screenshots of Key Pages

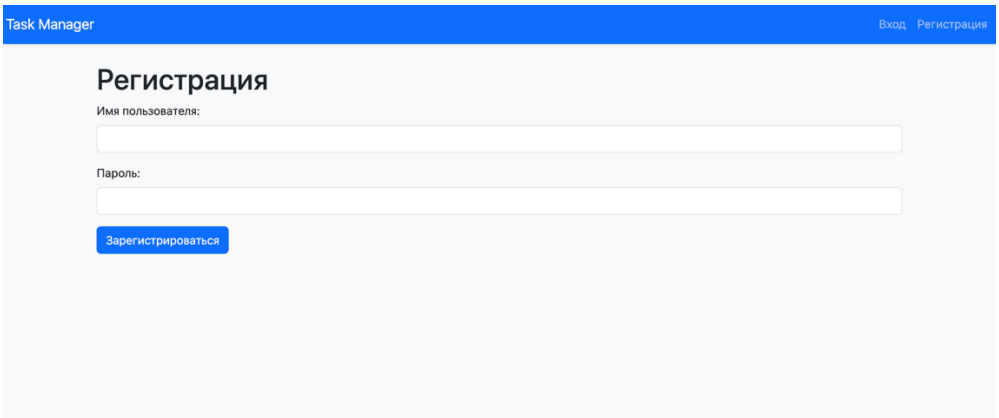
Login Page

- Description: A form for logging into the system (login.html). Users enter their username and password, with a "Login" button.



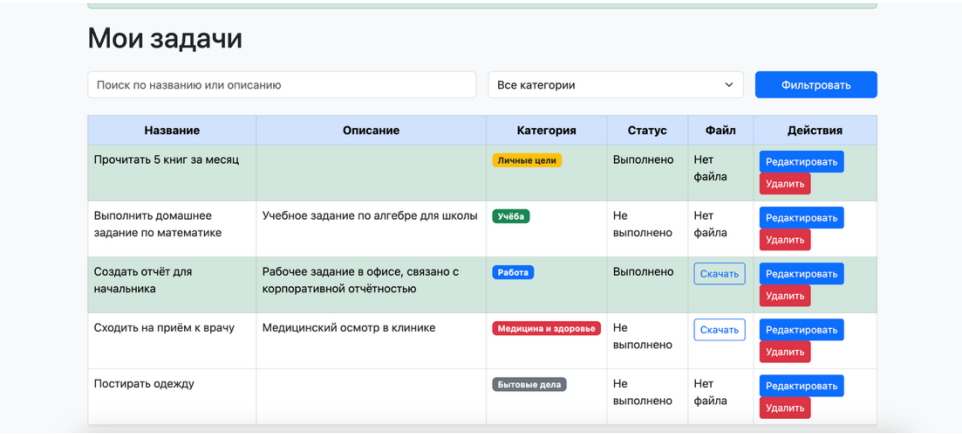
Registration Page

- Description: A form for registering a new user (register.html). Users provide a username and password, with a "Register" button.



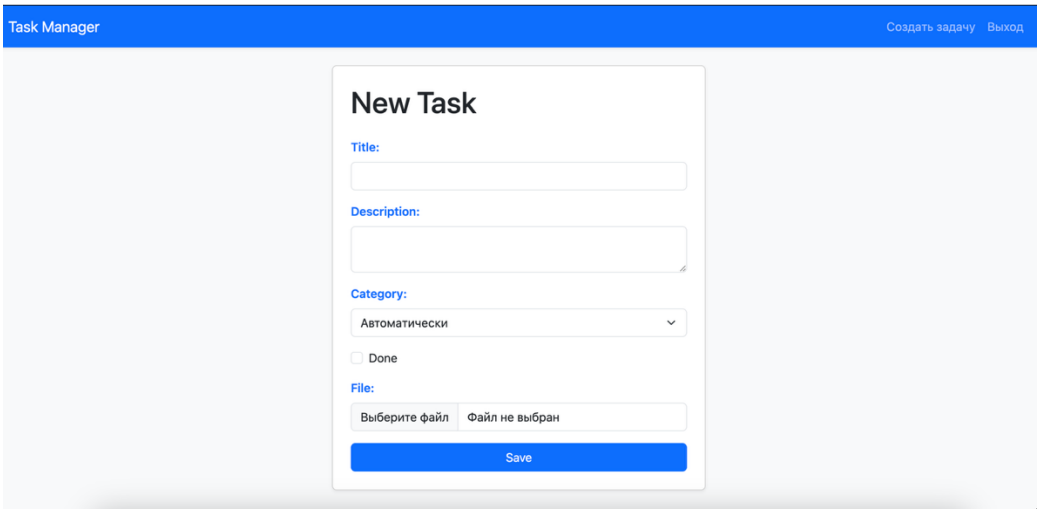
Task List Page

- Description: The main page (tasks.html) with a table of tasks. The table shows the title, description, category, status, file, and buttons for editing/deleting. It includes category filters and a search bar.



Task Form Page

- Description: A page for creating or editing a task (task_form.html). The form includes fields for title, description, category, status (done/not done), and file upload.



6, Challenges and Solutions

No File Type Validation

- **Issue**: There was no validation for uploaded file types, allowing potentially dangerous files (e.g., .exe).
- **Solution**: Added a function allowed_file and a list ALLOWED_EXTENSIONS = {'png', 'jpg', 'jpeg', 'gif', 'pdf'} in routes.py, along with validation before saving files.

Issues with AI (Model facebook/bart-large-mnli)

- **Issue**: The facebook/bart-large-mnli model, used for automatic task categorization, sometimes produces inaccurate results. For instance, in routes.py, there's logic to select the second category (result['labels'][1]) if "Household" is predicted with less than 50% confidence. However, this doesn't always work well—e.g., a task like "Submit Python exam" might be classified as "Work" instead of "Study." Additionally, the model is resource-heavy, and loading it (classifier = pipeline("zero-shot-classification", model="facebook/bart-large-mnli")) can strain smaller servers.
- Solution: Added error handling in routes.py. If categorization fails, the category is set to "Not Defined," and a message is shown to the user via flash('Categorization error: {str(e)}', 'warning'). Logging (logger.info) was also added to track categorization probabilities for future improvements. A potential future enhancement could involve replacing the model with a lighter alternative, such as using pre-trained embeddings with a simple classifier (e.g., via scikit-learn).

7. Conclusion

The "Task Manager" project has been successfully implemented and meets all requirements. It enables users to manage tasks, attach files, automatically categorize tasks, and ensures security (CSRF tokens, file restrictions). The interface is user-friendly thanks to Bootstrap and informative feedback (flash messages).

Possible Improvements:

- Add pagination for the task list to handle large numbers of tasks.
- Improve task categorization by replacing facebook/bart-large-mnli with a more accurate or lightweight model.
- Support additional file types (e.g., .docx) with appropriate validation.

8. References

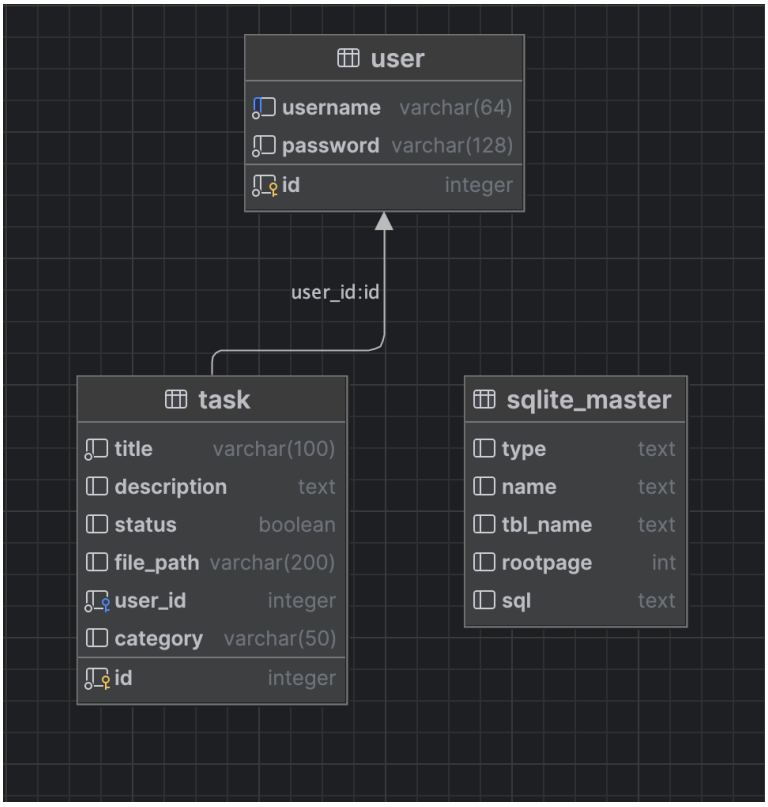
- **GitHub Repository**: https://github.com/Dina-45/todo_app/tree/main/app
- **Website**: http://kuralbaydina.pythonanywhere.com

- Input validation (e.g., checking field lengths, file types).
- Success and error messages are displayed using Bootstrap alerts.

2. Database Diagram:

The application's database uses SQLite and consists of two tables: user and task.

Diagram:



Relationships:one-to-many

3. Explanation of Code Structure (OOP and Blueprints)

The application is built using an object-oriented programming (OOP) approach:

Models (app/models.py):

Class User:

- Attributes: id, username, password, tasks (relationship with tasks).
- Methods: set_password (hashes the password), check_password (verifies the password).

Class Task:

- Attributes: id, title, description, status, file_path, user_id.
- Relationship with User via user_id.

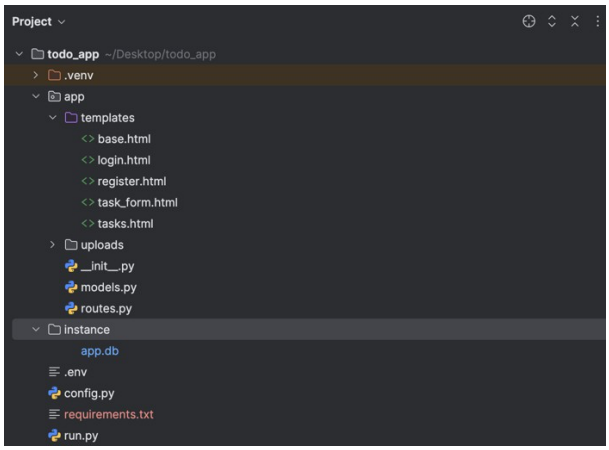
Modularity:

- The application is divided into modules using Flask Blueprints:

  auth_bp: Routes for authentication (/register, /login, /logout).

  tasks_bp: Routes for task management (/, /task/new, /task/<id>/edit, etc.).

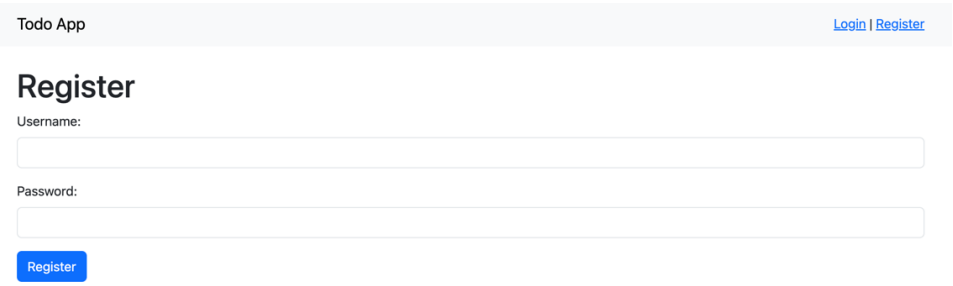
Project Structure:



4. Screenshots of Key Pages

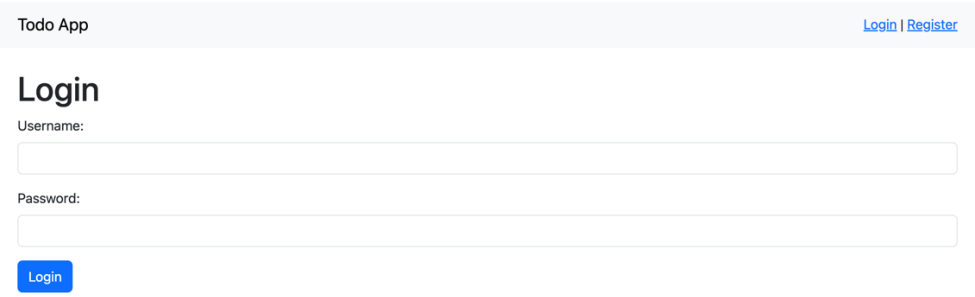
1. Registration Page

Description: A form for user registration with fields for username and password.



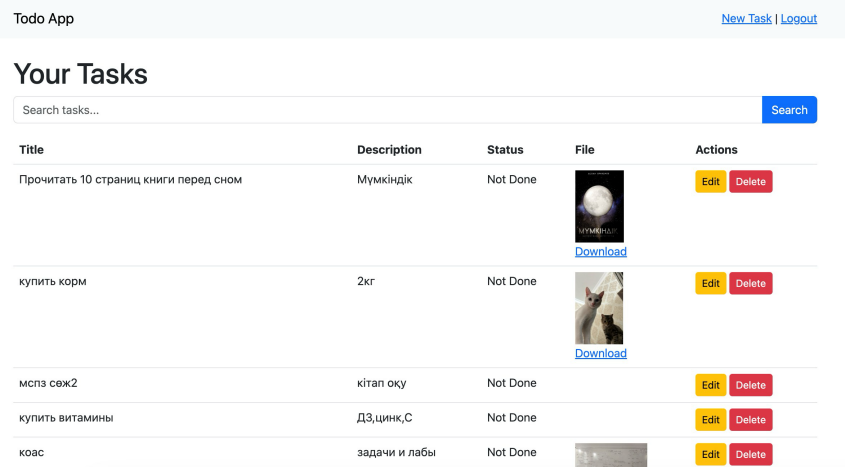
2. Login Page

Description: A form for logging into the system.



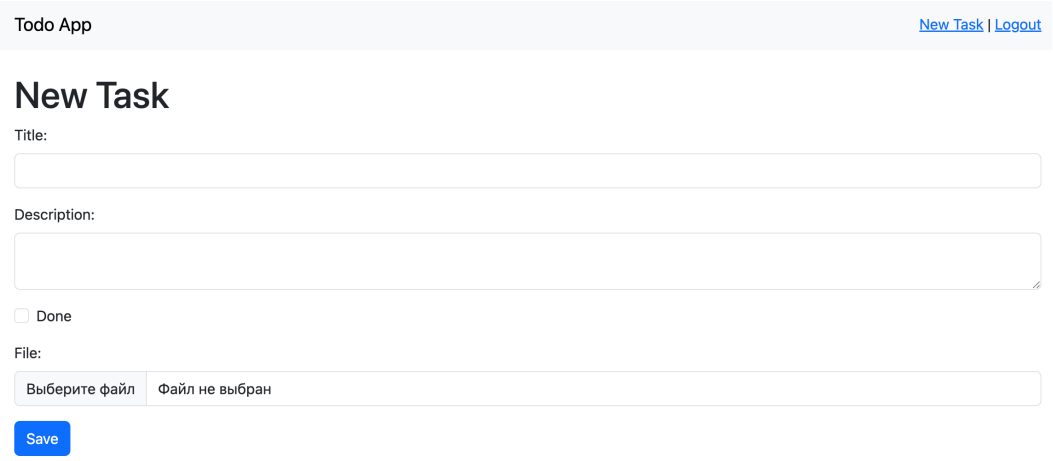
3. Tasks List Page

Description: A table displaying tasks, including title, description, status, image, and actions (edit/delete).



4. Task Creation Form

Description: A form for creating a new task with fields for title, description, status, and file upload.



5. Issues Encountered and Their Solutions

AttributeError: 'User' object has no attribute 'set_password'

Description: An error occurred during user registration because the set_password method was missing in the User class.

Cause: The set_password method was not defined in the User model, although it was called in the register route.

Solution: Added the set_password and check_password methods to the User class in app/models.py.

Not Found Error When Downloading Uploaded Files

Description: Clicking the "Download" link for an uploaded image resulted in a Not Found error, even though the file existed in the app/uploads folder.

Cause: Incorrect file path handling in the uploaded_file route due to improper use of send_from_directory.

Solution:

Fixed the path to the uploads folder using current_app.config['UPLOAD_FOLDER'].

Added debug output to verify the file path.

Separated functionality: the uploaded_file route for displaying files, and the download_file route for downloading with as_attachment=True.

Conclusion:

The app works well and does everything it was supposed to do. It's easy to use — you can add, edit, and delete tasks, and even upload pictures for them. I also made sure it checks the data and shows errors if something goes wrong. While making the app, we fixed some problems, which made it more stable. In the end, it turned out to be a useful and simple tool for managing tasks.